

OPERATING SYSTEMS
UE25CS242A

Deadlocks

Dr. Anitha K
Department of Computer Science and Engineering

Acknowledgement, Prof. Pavan A C

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course



- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources:

1. “Operating System Concepts”, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne 10th Edition, John Wiley & Sons, Global Edition, 2023.
2. “Advanced Programming in the Unix Environment”, Richard Stevens and Stephen A Rago, Pearson, 3rd edition, 2017.
3. Some conceptual text and diagram from “Operating Systems, Internals and Design Principles”, William Stallings, 9th Edition, Pearson, 2018.

OPERATING SYSTEMS

Principles, Detection, Prevention and Avoidance

Dr. Anitha K

Department of Computer Science and Engineering

Introduction

- Multiprogramming environment: several processes compete to limited number of resources.
- A Process is holding a resource(R1) and is waiting for the resources(R2).
- The resource R2 is held by another process.
- Waiting state of processes will not change, as the requested resource is held by the waiting process.
- This situation is called **deadlock**.

PES UNIVERSITY
CELEBRATING 50 YEARS

System Model

- System consists of finite number of resources.
- Resource types $R_1, R_2, \ldots, R_m$
  - *Physical resources: CPU cycles, memory space, I/O devices, printer, tape drives.*
  - *Logical resources: semaphores, mutex locks, files.*
- Each resource type R_i has W_i instances.
- Ex: if the system has 2 CPU's then CPU has 2 instances.
- Each process utilizes a resource as follows:
  - **Request :** Process makes request to the resource. Eg. system call like request(), open(), wait(),allocate(), etc.
  - **Use:** operates on these resources.
  - **Release:** process releases the resources. Eg. Using a system call like release(), close(), signal(), free etc.
- Request and release of semaphore, acquire and release of lock on mutex.

PES UNIVERSITY
CELEBRATING 50 YEARS

Deadlock with physical resources



Example 1

- Consider a system with 3 CD RW drives.
- Suppose 3 processes(p0,p1,p2) are holding one drive each.
- What happens,
  - If a process p0 makes a request for one more drive.

Example 2

- Consider a system with one printer one DVD drive.
- Process Pi is holding printer and process Pj is holding DVD drive.
- What happens if
  - Process Pi request DVD and Pj requests printer

Does dead lock occur in example 1 and 2?

Deadlock with Semaphores



- Data:
  - A semaphore **S1** initialized to 1
  - A semaphore **S2** initialized to 1
- Two processes P1 and P2
- **P1:**

wait(s1)

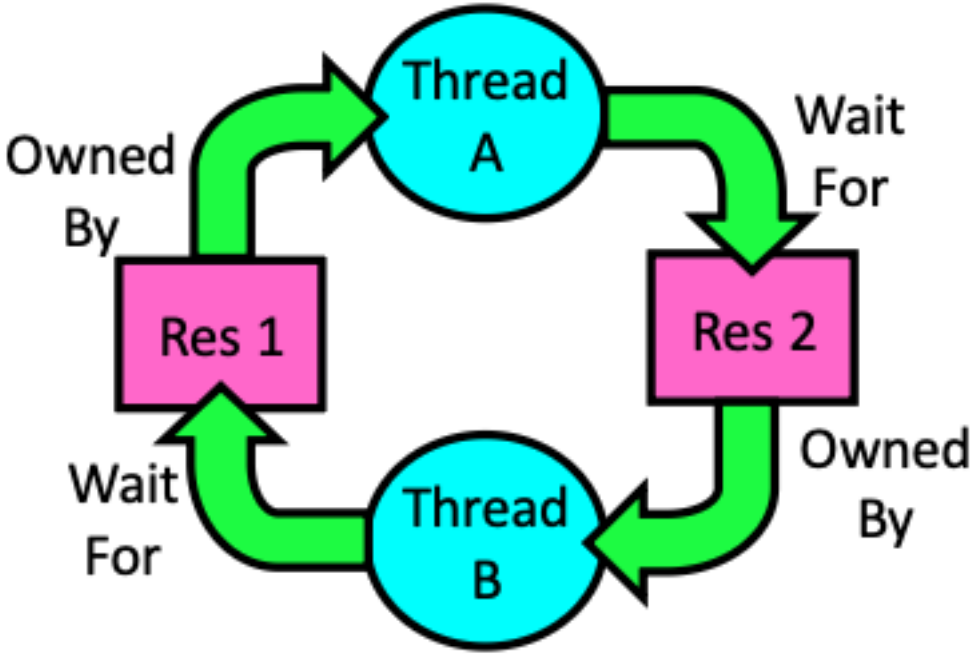
wait(s2)

- **P2:**

wait(s2)

wait(s1)

Deadlock: A Deadly type of Starvation

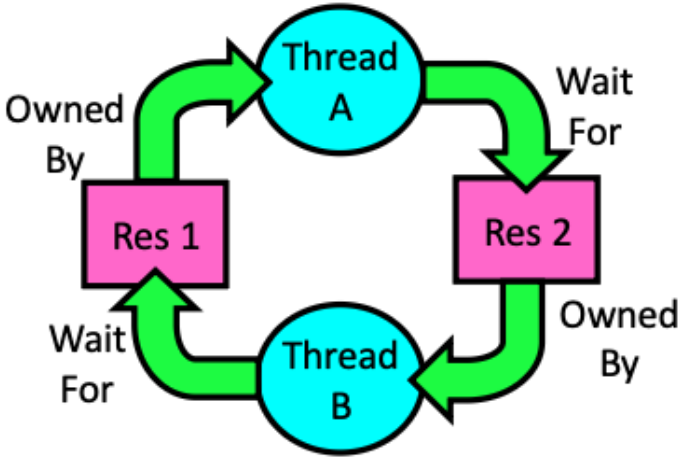


Deadlock: A Deadly type of Starvation

PES UNIVERSITY
CELEBRATING 50 YEARS

- Starvation: thread waits indefinitely
  - Example, low-priority thread waiting for resources as constantly in use by high-priority threads

- Deadlock: circular waiting for resources
  - Thread A owns Res 1 and is waiting for Res 2
  - Thread B owns Res 2 and is waiting for Res 1



- Deadlock - Starvation but not vice versa
  - Starvation can end (but doesn't have to)
  - Deadlock can't end without external intervention

OPERATING SYSTEMS

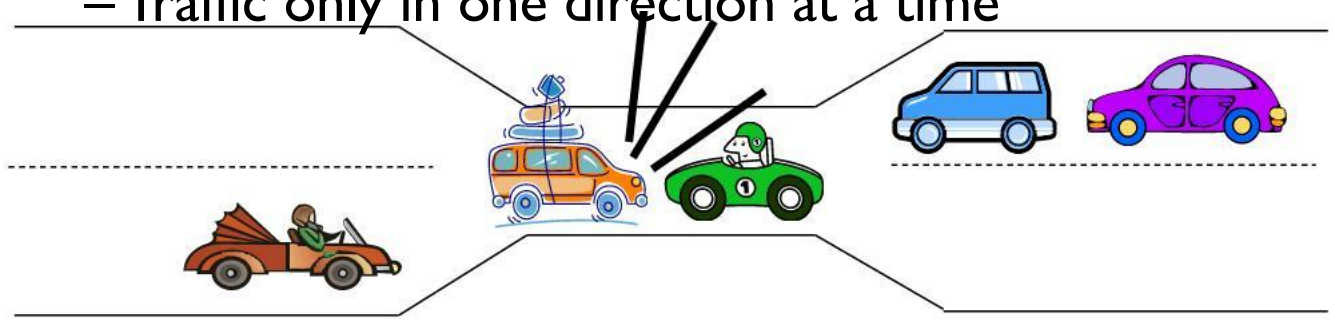
Example: Single-Lane Bridge Crossing



PES UNIVERSITY
CELEBRATING 50 YEARS

Bridge Crossing Example

- Each segment of road can be viewed as a resource
  - Car must own the segment under them
  - Must acquire segment that they are moving into
- For bridge: must acquire both halves
  - Traffic only in one direction at a time



- Deadlock: Shown above when two cars in opposite directions meet in middle
  - Each acquires one segment and needs next
  - Deadlock resolved if one car backs up (preempt resources and rollback)
    - » Several cars may have to be backed up
- Starvation (not Deadlock):
  - East-going traffic really fast - no one gets to go west

Deadlock with Locks

```
Thread A:                Thread B:
x.Acquire();             y.Acquire();
y.Acquire();             x.Acquire();
                                …
…
y.Release();             x.Release();
x.Release();             y.Release();
```

- This lock pattern exhibits *non-deterministic deadlock*
  - Sometimes it happens, sometimes it doesn't!
- This is really hard to debug!

Deadlock Characterization



Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion**: only one process at a time can use a resource (sharable resources like Read-only files do not require mutually exclusive access and thus cannot be involved in a deadlock).
- **Hold and wait**: a process holding at least one resource is waiting to acquire additional resources held by other processes.
- **No preemption**: a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- **Circular wait**: there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.

OPERATING SYSTEMS

Resource-Allocation Graph

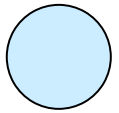
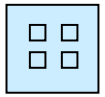
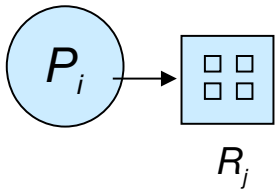
- Deadlocks are described precisely with directed graphs called system resource-allocation graph.

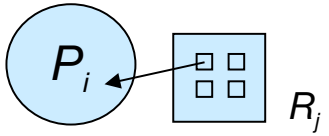
A graph consists of set of vertices *V* and a set of edges *E*.

- V is partitioned into two types:
  - $P = \{P_1, P_2, \ldots, P_n\}$, the set consisting of all the processes in the system
  - $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $P_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow P_i$

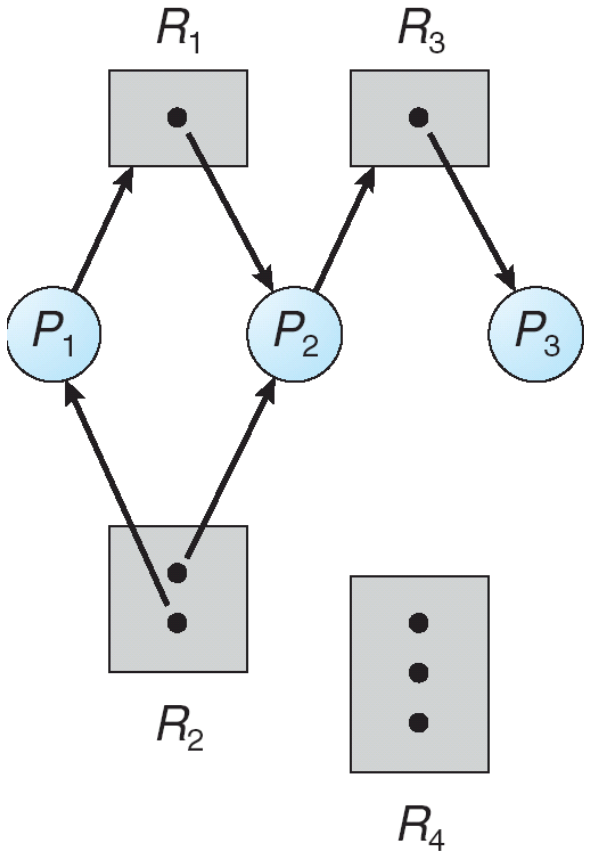
Resource-Allocation Graph (Cont.)

A set of vertices *V* and a set of edges *E*.

- Process 
- Resource Type with 4 instances 
- P_i requests instance of R_j 

- P_i is holding an instance of R_j 

Example of a Resource-Allocation Graph

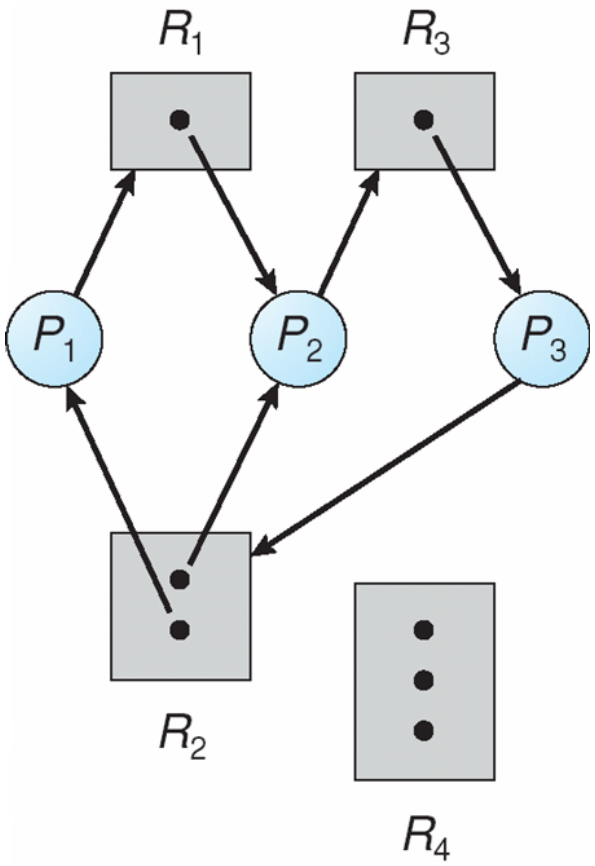


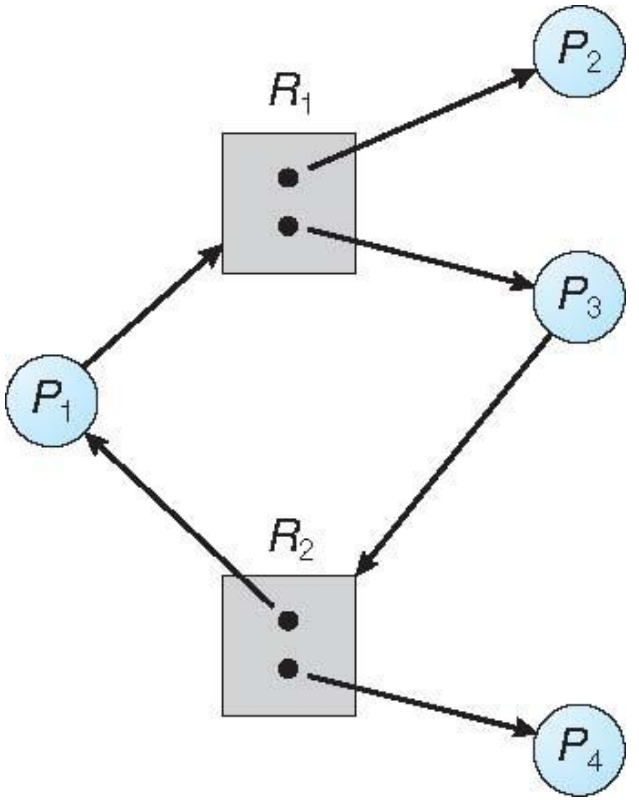
Set of process: P1,P2,P3

Set of Resources: R1,R2,R3

OPERATING SYSTEMS

Resource-Allocation Graph





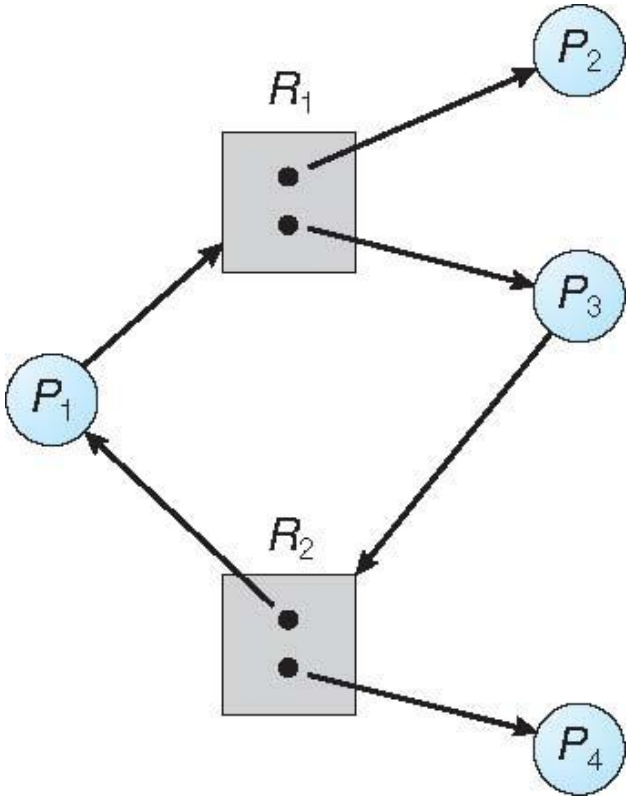
P1 -> R1 -> P2 -> R3 -> P3 -> R2 -> P1

P2 -> R3 -> P3 -> R2 -> P2

P1 -> R1 -> P3 -> R2 -> P1

OPERATING SYSTEMS

Resource-Allocation Graph



P1 -> R1 -> P3 -> R2 -> P1

Basic Facts

- If graph contains no cycles ⇒ no deadlock
- If graph contains a cycle ⇒
  - if only one instance per resource type, then deadlock
  - if several instances per resource type, the system may or may not be in a deadlocked state

PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Methods for Handling Deadlocks



- **Deadlock prevention**: write your code in a way that it isn't prone to deadlock.
- **Deadlock recovery**: let deadlock happen, and then figure out how to recover from it.
- **Deadlock avoidance**: dynamically delay resource requests so deadlock doesn't happen.
- **Deadlock denial**: ignore the possibility of deadlock.

- Modern operating systems:
  – Make sure the system isn't involved in any deadlock
  – Ignore deadlock in applications » "Ostrich Algorithm"

Deadlock Prevention

4 Necessary conditions for deadlock to occur:

- **Mutual Exclusion**
  - At least one resource must be non sharable.
    - Ex. printers and tape drives, mutex locks
  - Sharable resources do not require mutual exclusion.
    - Ex . Read-only files
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources.
  - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none allocated to it.
  - Low resource utilization; starvation possible.

PES UNIVERSITY
CELEBRATING 50 YEARS

Deadlock Prevention (Cont.)



- **No Preemption** –
  - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released.
  - Preempted resources are added to the list of resources for which the process is waiting.
  - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.
- **Circular Wait** –
  - Each resource will be assigned with a numerical number.
  - A process can request the resources increasing/decreasing order of numbering.

Techniques for Preventing Deadlock

- Make all threads request everything they'll need at the beginning.
  - Problem: Predicting future is hard, tend to over-estimate resources
  - Example:
    - » If need 2 chopsticks, request both at same time

- Force all threads to request resources in a particular order preventing any cyclic use of resources
  - Thus, preventing deadlock
  - Example (`x.Acquire()`, `y.Acquire()`, `z.Acquire()`,…)
    - » Make tasks request disk, then memory, then…
    - » Keep from deadlock on freeways around SF by requiring everyone to go clockwise

OPERATING SYSTEMS

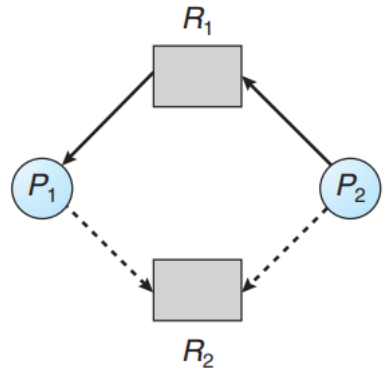
Deadlock Detection

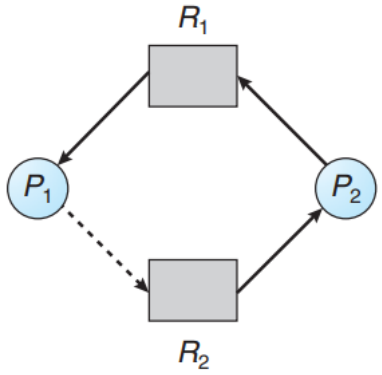


- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

Single Instance of Each Resource Type

- In a resource allocation graph, a claim edge $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j at some time in the future. This edge is represented in the graph by a dashed line.
- Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock.
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph.

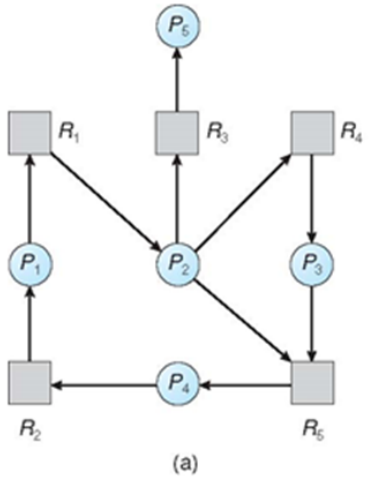




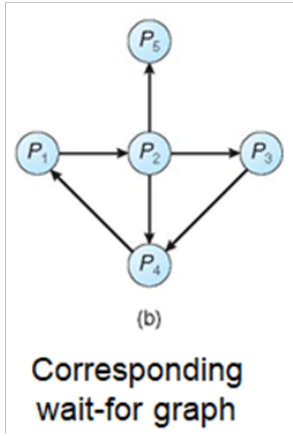
OPERATING SYSTEMS

Resource-Allocation Graph and Wait-for Graph

- If all resources have only a single instance, then we can define a deadlock detection algorithm that uses a variant of the resource-allocation graph, called a wait-for graph.
- We obtain wait-for graph from the resource-allocation graph by removing the resource nodes and collapsing the appropriate edges.
- An edge from P_i to P_j implies that process P_i is waiting for process P_j to release a resource that P_i needs.
- An edge $P_i \rightarrow P_j$ exists in a wait-for graph if and only if the corresponding resource allocation graph contains two edges $P_i \rightarrow R_q$ and $R_q \rightarrow P_j$ for some resource R_q
- A **deadlock exists** in the system if and only if the **wait-for graph contains a cycle**.
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph.



Resource-Allocation Graph



Corresponding wait-for graph

OPERATING SYSTEMS

Deadlock Avoidance



- Deadlock-prevention algorithms, prevent deadlocks by limiting how requests can be made.
- The limits ensure that at least one of the necessary conditions for deadlock cannot occur.
- Possible side effects of preventing deadlocks by this method, however, are low device utilization and reduced system throughput.
- **Deadlock avoidance** requires that the operating system be given additional information in advance concerning which resources a process will request and use during its lifetime.
- With this additional knowledge of complete sequence of requests and releases for each process, the system can decide for each request whether or not the process should wait in order to avoid a possible future deadlock.
- To decide whether the current request can be satisfied or must be delayed, the system must consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process.

OPERATING SYSTEMS

Deadlock Avoidance



- The simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need.
- Given this a priori information, it is possible to construct an algorithm that ensures that the system will never enter a deadlocked state.
- A deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular-wait condition can never exist.
- The resource allocation ***state*** is defined by the number of available and allocated resources and the maximum demands of the processes.

OPERATING SYSTEMS

Safe State



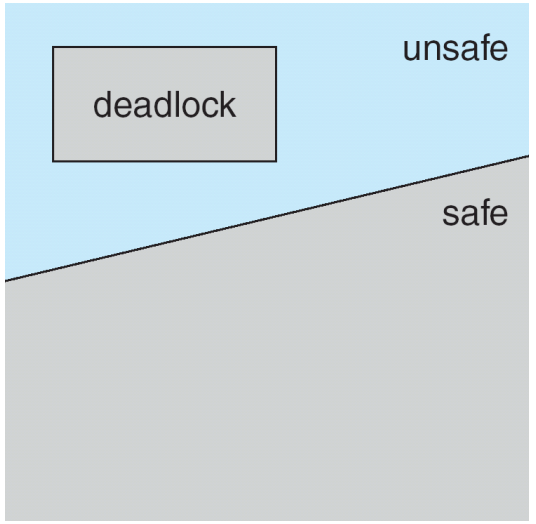
- A state is ***safe*** if the system can allocate resources to each process (up to its maximum) in some order and still avoid a deadlock.
- More formally, a system is in a safe state only if there exists a **safe sequence**.
- A sequence of processes <*P*1, *P*2, ..., *Pn*> is a safe sequence for the current allocation state if, for each *Pi* , the resource requests that *Pi* can still make can be satisfied by the currently available resources plus the resources held by all *Pj*, with $j < i$.
- In this situation, if the resources that *Pi* needs are not immediately available, then *Pi* can wait until all *Pj* have finished.
- When they have finished, *Pi* can obtain all of its needed resources, complete its designated task, return its allocated resources, and terminate.
- When *Pi* terminates, *Pi*+1 can obtain its needed resources, and so on. If no such sequence exists, then the system state is said to be ***unsafe.***

OPERATING SYSTEMS

Safe, Unsafe and Deadlock States



- A safe state is not a deadlocked state.
- Conversely, a deadlock state is an unsafe state.
- Not all unsafe states are deadlocks.
- An unsafe state may lead to a deadlock.
- As long as the state is safe, the OS can avoid unsafe states.
- In an unsafe state, the OS cannot prevent processes from requesting resources in such a way that a deadlock occurs.
- The behavior of processes controls unsafe states.
- If a system is in safe state → no deadlocks
- If a system is in unsafe state → possibility of deadlock
- Goal for Avoidance → ensure that a system will never enter an unsafe state.



OPERATING SYSTEMS

Deadlock avoidance algorithms



- Avoidance algorithms ensure that the system will never deadlock.
- The idea is simply to ensure that the system will always remain in a safe state.
- Initially, the system is in a safe state. Whenever a process requests a resource that is currently available, the system must decide whether the resource can be allocated immediately or whether the process must wait.
- The request is granted only if the allocation leaves the system in a safe state.
- In this scheme, if a process requests a resource that is currently available, it may still have to wait.
- Thus, resource utilization may not be optimal.

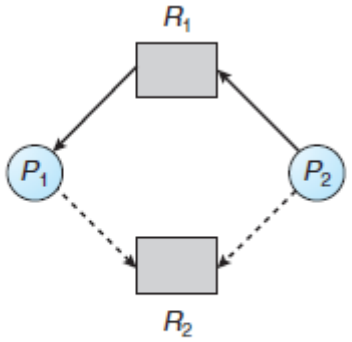
OPERATING SYSTEMS

Resource Allocation Graph algorithm

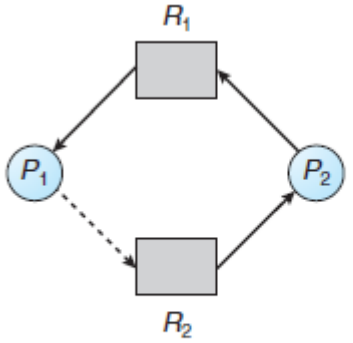
- In addition to the request and assignment edges in a resource allocation graph, we introduce a new type of edge, called a **claim edge.**
- A claim edge *Pi* → *Rj* indicates that process *Pi* may request resource *Rj* at some time in the future. This edge resembles a request edge in direction but is represented in the graph by a dashed line.
- When process *Pi* requests resource *Rj* , the claim edge *Pi* → *Rj* is converted to a request edge. Similarly, when a resource *Rj* is released by *Pi* , the assignment edge *Rj* → *Pi* is reconverted to a claim edge *Pi* → *Rj*
- The resources must be claimed a priori in the system. That is, before process *Pi* starts executing, all its claim edges must already appear in the resource-allocation graph.
- Now suppose that process *Pi* requests resource *Rj*. The request can be granted only if converting the request edge *Pi* → *Rj* to an assignment edge *Rj* → *Pi* does not result in the formation of a cycle in the resource-allocation graph.

OPERATING SYSTEMS

Resource Allocation Graph algorithm example



Resource-allocation graph for deadlock avoidance.



An unsafe state in a resource-allocation graph.

- Consider the above resource-allocation graph.
- Suppose that *P*2 requests *R*2. Although *R*2 is currently free, we cannot allocate it to *P*2, since this action will create a cycle in the graph below.
- A cycle indicates that the system is in an unsafe state.
- If *P*1 requests *R*2, then a deadlock will occur.

OPERATING SYSTEMS

Banker's algorithm



- When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need. This number may not exceed the total number of resources in the system.
- When a user requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state. If it will, the resources are allocated; otherwise, the process must wait until some other process releases enough resources.
- Several data structures must be maintained to implement the banker's algorithm. These data structures encode the state of the resource-allocation system.

Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

Available: Vector of length m. If available [j] = k, there are k instances of resource type R_j available

Max: n x m matrix. If Max [i,j] = k, then process P_i may request at most k instances of resource type R_j

Allocation: n x m matrix. If Allocation[i,j] = k then P_i is currently allocated k instances of R_j

Need: n x m matrix. If $Need$[i,j] = k, then P_i may need k more instances of R_j to complete its task

$Need$ [i,j] = Max[i,j] – $Allocation$ [i,j]

OPERATING SYSTEMS

Safety Algorithm



1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively. Initialize:

   Work* = *Available

   ***Finish* [*i*] = *false* for *i* = 0, 1, …, *n*- 1**

2. Find an ***i*** such that both:

   (a) ***Finish* [*i*] = *false***

   (b) $Need_i \leq Work$

   If no such ***i*** exists, go to step 4

3. $Work = Work + Allocation_i$

   Finish*[*i*] = *true

   go to step 2

4. If ***Finish* [*i*] == *true*** for all ***i***, then the system is in a safe state

Resource-Request Algorithm for Process Pi

Request$_i$ = request vector for process P_i. If ***Request***$_i$**[*j*] = *k*** then process P_i wants ***k*** instances of resource type R_j

1. If ***Request***$_i \leq$ ***Need***$_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If ***Request***$_i \leq$ ***Available***, go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

- If safe ⇒ the resources are allocated to P_i
- If unsafe ⇒ P_i must wait, and the old resource-allocation state is restored

Example of Banker's Algorithm



5 processes P0 through P4;

3 resource types: A (10 instances), B (5instances), and C (7 instances)

Snapshot at time T0:

	Allocation	Max	Available
	A B C	A B C	A B C
P0	0 1 0	7 5 3	3 3 2
P1	2 0 0	3 2 2	
P2	3 0 2	9 0 2	
P3	2 1 1	2 2 2	
P4	0 0 2	4 3 3	

Example (Cont.)

The content of the matrix **Need** is defined to be **Max – Allocation**

	Need
	A B C
P0	7 4 3
P1	1 2 2
P2	6 0 0
P3	0 1 1
P4	4 3 1

The system is in a safe state since the sequence < P1, P3, P4, P2, P0> satisfies safety criteria

OPERATING SYSTEMS

Example: P1 Request (1,0,2)

- Check that Request ≤ Available (that is, (1,0,2) ≤ (3,3,2) ⇒ true

	Allocation	*Need*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Executing safety algorithm shows that sequence < P_1, P_3, P_4, P_0, P_2> satisfies safety requirement
- Can request for (3,3,0) by P_4 be granted?
- Can request for (0,2,0) by P_0 be granted?



THANK YOU

Dr. Anitha K

Department of Computer Science and Engineering

anithak@pes.edu